

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

-----o0o-----



BÁO CÁO ĐỒ ÁN LAB 3: PAGE TABLES

MÔN: HỆ ĐIỀU HÀNH

Giảng viên lý thuyết: Trần Trung Dũng

Giảng viên thực hành: Nguyễn Thanh Quân

Hồ Chí Minh, ngày 02 tháng 01 năm 2025

THÔNG TIN NHÓM

MSSV	Họ và tên	Phân công công việc
22120091	Phạm Khánh Hân	Speed up system calls
22120094	Lê Bảo Hồng Hạnh	Print a page table
22120213	Đoàn Thị Minh Anh	Detect which pages have been accessed

Mục lục

I. Speed up system calls	4
1. Mục tiêu:	4
2. Quy trình thực hiện:	4
3. Trả lời câu hỏi thêm:	4
II. Print a page table.....	5
1. Mục tiêu:	5
2. Quy trình thực hiện:	5
3. Trả lời câu hỏi thêm:	5
III. Detect which pages have been accessed	7
1. Mục tiêu:	7
2. Quy trình thực hiện:	7
3. Kết quả đạt được:	8

I. Speed up system calls

1. Mục tiêu:

Thay vì thực hiện một hệ thống lệnh gọi đầy đủ (yêu cầu chuyển đổi ngữ cảnh từ user space sang kernel và ngược lại), ta có thể sử dụng bộ nhớ chia sẻ để truy cập nhanh thông tin như PID mà không cần ngắt (interrupt) hay chuyển ngữ cảnh.

2. Quy trình thực hiện:

- Trong file kernel/proc.h tại struct proc, thêm con trỏ struct usyscall để quản lý và lưu địa chỉ của page được chia sẻ với không gian người dùng → kernel biết địa chỉ vùng bộ nhớ được ánh xạ, từ đó truy cập và cập nhật các giá trị liên quan (như PID).
- Trong file kernel/proc.c tại:
 - o Ở hàm allocproc, thêm đoạn mã để cấp phát một page mới (usyscall page) và liên kết nó với struct usyscall *usyscall của tiến trình.
 - o Ở hàm proc_pagetable, ánh xạ page chia sẻ vào không gian bộ nhớ ngay bên dưới vùng trampoline (vùng bộ nhớ đặc biệt dùng để chuyển đổi giữa user space và kernel).
 - o Ở hàm freeproc, thêm logic để giải phóng vùng bộ nhớ của usyscall page khi tiến trình bị hủy.
 - o Ở hàm proc_freepagetable, thêm logic để xóa ánh xạ (unmap) usyscall page trước khi giải phóng bảng trang.

3. Trả lời câu hỏi thêm: Những lệnh gọi hệ thống nào khác trong xv6 có thể được tăng tốc bằng cách sử dụng trang chia sẻ này? Hãy giải thích.

- **uptime()**: Là lệnh gọi hệ thống trả về thời gian hoạt động hiện tại của hệ thống. Giá trị này thay đổi liên tục, kernel có thể cập nhật giá trị này vào trang chia sẻ ở các khoảng thời gian cố định (ví dụ: mỗi 10 mili giây).

Cách thực hiện:

- Thêm một trường timeticks vào cấu trúc usyscall để lưu trữ số lượng ticks của đồng hồ hệ thống.
- Trong hàm clockintr() của kernel, cập nhật giá trị mới nhất của timeticks vào trang chia sẻ khi có sự kiện ngắt đồng hồ.
- Trong không gian người dùng, khi gọi uptime (phiên bản tối ưu hóa của uptime), tiến trình chỉ cần truy cập giá trị timeticks trực tiếp từ trang chia sẻ (USYSCALL Page) mà không cần thực hiện chuyển đổi ngữ cảnh qua kernel.

II. Print a page table

1. Mục tiêu:

Giúp hình dung về các bảng trang trong hệ thống RISC-V và có thể hỗ trợ cho việc gỡ lỗi trong tương lai.

2. Quy trình thực hiện:

Bước 1: Tạo hàm in bảng trang (vmprint)

- Viết hàm **vmprintRecursive** và **vmprint** trong file kernel/vm.c:
 - vmprintRecursive:** Duyệt đệ quy qua các cấp của bảng trang (Level 0, Level 1, Level 2).
 - In các mục bảng trang (PTE) hợp lệ, địa chỉ vật lý ánh xạ và kiểm tra tiếp các bảng trang cấp dưới nếu cần.
- Mỗi cấp độ bảng trang được in ra với dấu ".." để thể hiện cấp độ.

Bước 2: Khai báo hàm vmprint

- Thêm prototype của hàm **vmprint** vào file kernel/defs.h để có thể sử dụng trong các file kernel khác.

Bước 3: Gọi hàm vmprint trong tiến trình đầu tiên

- Sửa file kernel/exec.c:
 - Trong hàm **exec**, kiểm tra nếu tiến trình là tiến trình đầu tiên (pid == 1), gọi **vmprint** để in bảng trang của tiến trình đó.

Bước 4: Biên dịch và kiểm tra

- Chạy lệnh `make qemu` để khởi động xv6 và kết quả đầu ra như sau:

```
page table 0x0000000087f6b000
..0: pte 0x0000000021fd9c01 pa 0x0000000087f67000
.. ..0: pte 0x0000000021fd9801 pa 0x0000000087f66000
.. .. ..0: pte 0x0000000021fda01b pa 0x0000000087f68000
.. .. ..1: pte 0x0000000021fd9417 pa 0x0000000087f65000
.. .. ..2: pte 0x0000000021fd9007 pa 0x0000000087f64000
.. .. ..3: pte 0x0000000021fd8c17 pa 0x0000000087f63000
..255: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..511: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..509: pte 0x0000000021fdcc13 pa 0x0000000087f73000
.. .. ..510: pte 0x0000000021fdd007 pa 0x0000000087f74000
.. .. ..511: pte 0x0000000020001c0b pa 0x0000000080007000
init: starting sh
```

3. Trả lời câu hỏi thêm:

Giải thích chi tiết về từng mục leaf page:

1. Leaf Page: **pte 0x0000000021fda01b pa 0x0000000087f68000**

- Nội dung logic: Trang này chứa dữ liệu chỉ đọc hoặc mã chương trình.
- Các bit quyền:

Chuyển đổi PTE sang dạng nhị phân: 10000111111101101000000001**1011**

- (Bit 0) PTE_V: Hợp lệ.
- (Bit 1) PTE_R: Cho phép đọc.
- (Bit 2) PTE_W: Không cho phép ghi.
- (Bit 3) PTE_X: Cho phép thực thi.

2. Leaf Page: **pte 0x0000000021fd9417 pa 0x0000000087f65000**

- Nội dung logic: Một trang dữ liệu đọc/ghi.
- Các bit quyền:

Chuyển đổi PTE sang dạng nhị phân: 10000111111101100101000001**1011**

- PTE_V: Hợp lệ.
- PTE_R: Cho phép đọc.
- PTE_W: Cho phép ghi.
- PTE_X: Không cho phép thực thi.

3. Leaf Page: **pte 0x0000000021fd9007 pa 0x0000000087f64000**

- Nội dung logic: Một trang dữ liệu đọc/ghi.
- Các bit quyền:

Chuyển đổi PTE sang dạng nhị phân: 10000111111101100100000000**1111**

- PTE_V: Hợp lệ.
- PTE_R: Cho phép đọc.
- PTE_W: Cho phép ghi.
- PTE_X: Không cho phép thực thi.

4. Leaf Page: **pte 0x0000000021fd8c17 pa 0x0000000087f63000**

- Nội dung logic: Một trang dữ liệu đọc/ghi.
- Các bit quyền:

Chuyển đổi PTE sang dạng nhị phân: 10000111111101100011000001**1011**

- PTE_V: Hợp lệ.
- PTE_R: Cho phép đọc.
- PTE_W: Cho phép ghi.
- PTE_X: Không cho phép thực thi.

5. Leaf Page: **pte 0x0000000021fdcc13 pa 0x0000000087f73000**

- Nội dung logic: Một trang chỉ đọc, có thể chứa dữ liệu bất biến hoặc ánh xạ đến thư viện dùng chung.
- Các bit quyền:

Chuyển đổi PTE sang dạng nhị phân: 1000011111110110011000001**1001**

- PTE_V: Hợp lệ.
- PTE_R: Cho phép đọc.

- PTE_W: Không cho phép ghi.
 - PTE_X: Không cho phép thực thi.
6. Leaf Page: **pte 0x0000000021fdd007 pa 0x0000000087f74000**
- Nội dung logic: Một trang dữ liệu đọc/ghi.
 - Các bit quyền:
- Chuyển đổi PTE sang dạng nhị phân: 10000111111011101000000000111
- PTE_V: Hợp lệ.
 - PTE_R: Cho phép đọc.
 - PTE_W: Cho phép ghi.
 - PTE_X: Không cho phép thực thi.
7. Leaf Page: **pte 0x0000000020001c0b pa 0x0000000080007000**
- Nội dung logic: Một trang chứa mã chương trình hoặc dữ liệu chỉ đọc.
 - Các bit quyền:
- Chuyển đổi PTE sang dạng nhị phân: 1000000000000000001110000001011
- PTE_V: Hợp lệ.
 - PTE_R: Cho phép đọc.
 - PTE_W: Không cho phép ghi.
 - PTE_X: Có thể thực thi.

III. Detect which pages have been accessed

1. Mục tiêu:

- Cài đặt lệnh gọi hệ thống **pgaccess** để xác định các trang bộ nhớ đã được truy cập thông qua việc kiểm tra bit truy cập trong các mục bảng trang (PTE).
- Trả về thông tin này dưới dạng bitmask cho người dùng.

2. Quy trình thực hiện:

a) Định nghĩa bit truy cập:

- Bit truy cập PTE_A được định nghĩa là bit thứ 6 (giá trị 0x40) trong mục bảng trang (PTE).
- Định nghĩa được thêm vào tệp *kernel/riscv.h*: **#define PTE_A (1L << 6)**

b) Thêm lệnh gọi hệ thống pgaccess: (Phần này đã được thêm sẵn khi thực hiện *chuyển sang nhánh pgtbl*)

- Khai báo lệnh gọi hệ thống trong *kernel/syscall.h*:
#define SYS_pgaccess 30
- Cập nhật danh sách lệnh gọi trong *kernel/syscall.c*:

```
extern uint64 sys_pgaccess(void);  
[SYS_pgaccess] sys_pgaccess,
```

c) Triển khai hàm sys_pgaccess:

- Trong *kernel/sysproc.c*, viết hàm **int sys_pgaccess(void)**
 - Lấy tham số từ không gian người dùng: Sử dụng các hàm tiện ích **argaddr()** và **argint()** để nhận tham số từ người dùng. Trong đó: **base** là địa chỉ bắt đầu, **len** là số lượng trang cần kiểm tra, **mask** là địa chỉ bitmask trong không gian người dùng.
 - Kiểm tra giới hạn số lượng trang: Đảm bảo số lượng trang không nhỏ hơn 0 và không lớn hơn 64 (64 là số lượng bit tối đa của kiểu `uint64` – kiểu được hệ thống dùng để lưu trữ bitmask).
 - Khởi tạo **bitmask**.
 - Duyệt qua từng trang: Duyệt qua từng trang bắt đầu từ **base**, tính địa chỉ ảo của trang thứ *i*.
 - Lấy PTE của trang: Sử dụng hàm **walk()** để tìm PTE của trang.
 - Kiểm tra và xử lý bit PTE_A: Kiểm tra tính hợp lệ của bit PTE. Tiếp theo, nếu bit PTE_A được bật, đánh dấu bit tương ứng trong **bitmask** và xóa bit PTE_A.
 - Sao chép bitmask sang không gian người dùng: Sau khi duyệt xong, sử dụng **copyout()** để chuyển **bitmask** từ kernel sang địa chỉ bộ nhớ người dùng.

d) Kiểm tra:

- Hàm **pgaccess_test** trong *user/pgtbltest.c* được sử dụng để kiểm tra.
- Sử dụng **vmprint()** để in ra trạng thái bảng trang và xác minh rằng các bit PTE_A được xử lý đúng cách.

3. Kết quả đạt được:

- Chạy chương trình **pgtbltest** thành công.
- Các bitmask trả về sau **vmprint()** khớp với trạng thái truy cập của các trang bộ nhớ.